

Онлайн-оценка стоимости и длительности эволюционного программного поиска с использованием БЯМ

Краткий инженерно-экспериментальный отчёт по проекту AIRI №,29

Автор: Тупицин Станислав

Менторы: Ольга Волкова, Владимир Шапошников, Никита Глазков

Результаты работы по состоянию на 1 августа 2026 года

Аннотация

Разработана система онлайн-оценки потребления токенов и длительности эволюционного программного поиска с большими языковыми моделями (БЯМ). Решение объединяет стадийную телеметрию, робастную степенную экстраполяцию, измерение фактического параллелизма, калибровку остаточного времени и диагностического БЯМ-агента. На восьми завершённых исторических запусках медианная абсолютная ошибка длительности снизилась с 21% до 13% на 25% прогресса и с 37% до 11% на 50%; терминальная ошибка токенов уменьшилась с 22% до 7%. На семи завершённых парных задачах AlphaEvolve подтверждающего преимущества агента не обнаружено. Разведочный анализ пробуждений дал $p \approx 0,060$ после поправки Бонферрони за четыре окна. Основной результат проекта — воспроизводимый детерминированный оценщик и инфраструктура для быстрой проверки агентных политик.

Ключевые слова: эволюционный программный поиск, большие языковые модели, прогноз стоимости, длительность, робастная регрессия, AlphaEvolve.

1 Введение

В системах эволюционного программного поиска БЯМ многократно генерирует программы-кандидаты, после чего они исполняются, оцениваются и отбираются. FunSearch продемонстрировал применимость такого цикла к математическим задачам [1], а AlphaEvolve распространил подход на научную и алгоритмическую оптимизацию [2]. Открытый фреймворк GigaEvo предоставляет модульную реализацию гибридного БЯМ-эволюционного подхода и инфраструктуру для воспроизводимых экспериментов [3]. Стоимость запуска нерегулярна: длины запросов меняются, стадии срабатывают с различной частотой, вызовы повторяются, а асинхронные рабочие процессы дают переменный параллелизм.

Цель проекта — по наблюдаемой части запуска оценить итоговое число токенов $\hat{T}_N$ и полное календарное время $\hat{D}_N$. Работа относится к задачам прогнозирования при фиксированном вычислительном бюджете [4] и дополняет методы адаптивного распределения ресурсов [5]. Реализованы сбор событий, математический оценщик, диагностический агент, воспроизведение исторических потоков и экспериментальный контур.

Модель опирается на робастную оценку Тейла–Сена [6], поправку обратного логарифмического преобразования Дюана [7] и разложение длительности на сервисную нагрузку и достигнутый параллелизм, согласующееся с принципом закона Литтла [8].

2 Постановка задачи

Пусть запланировано N попыток мутации, а к моменту прогноза наблюдаются первые k попыток; доля прогресса равна $q = k/N$. Каждый вызов j относится к стадии s и содержит входные токены t_j^{in} , выходные токены t_j^{out} и задержку ℓ_j . Для бакета i стадии s определим

$$y_{s,i} = \sum_{j \in \mathcal{C}_{s,i}} (t_j^{\text{in}} + t_j^{\text{out}}), \quad o_{s,i} = \sum_{j \in \mathcal{C}_{s,i}} t_j^{\text{out}}. \quad (1)$$

Здесь $\mathcal{C}_{s,i}$ — множество модельных вызовов бакета. Основной инвариант оценителя:

$$\text{итог} = \text{точно измеренная часть} + \text{прогноз неизвестного хвоста}. \quad (2)$$

Таким образом, выполненная работа никогда не прогнозируется повторно.

3 Математическая модель

3.1 Частота стадий и токенный хвост

Число вызовов стадии не обязано совпадать с числом попыток. Если $n_s(k)$ — число бакетов стадии k попытке k , а $h = \lfloor k/2 \rfloor$, локальная частота оценивается свежей половиной наблюдений:

$$\hat{r}_s(k) = \frac{n_s(k) - n_s(h)}{k - h}, \quad \widehat{M}_s(N) = n_s(k) + (N - k)\hat{r}_s(k). \quad (3)$$

При малом числе точек используется накопленная частота $n_s(k)/k$. Величина $\widehat{M}_s$ задаёт прогноз общего числа бакетов стадии.

Для их размера принимается локальная степенная модель

$$y_s(x) = a_s x^{b_s}, \quad \log y_{s,i} = \log a_s + b_s \log i + \varepsilon_i. \quad (4)$$

Робастная оценка Тейла–Сена имеет вид [6]

$$\hat{b}_s = \text{median}_{i < j} \frac{\log y_{s,j} - \log y_{s,i}}{\log j - \log i}, \quad \log \hat{a}_s = \text{median}_i (\log y_{s,i} - \hat{b}_s \log i). \quad (5)$$

Для устранения занижения после обратного логарифмического преобразования используется множитель Дюана [7]:

$$e_i = \log y_{s,i} - (\log \hat{a}_s + \hat{b}_s \log i), \quad \hat{\kappa}_s = \frac{1}{m_s} \sum_{i=1}^{m_s} \exp(e_i). \quad (6)$$

Тогда прогноз неизвестного токенового хвоста стадии равен

$$\hat{R}_s^{(T)} = \sum_{x=m_s+1}^{\lfloor \widehat{M}_s \rfloor} \hat{\kappa}_s \hat{a}_s x^{\hat{b}_s}, \quad (7)$$

а итоговая оценка сохраняет наблюдаемую сумму точно:

$$\hat{T}_N = \underbrace{\sum_s \sum_{i=1}^{m_s} y_{s,i}}_{T_{\text{obs}}} + \sum_s \text{clip} \left(\hat{R}_s^{(T)}, 0, \text{cap}_s \right). \quad (8)$$

Ограничение cap_s защищает ранний прогноз от неограниченной степенной экстраполяции.

3.2 Сервисное и календарное время

Задержка модельного вызова раскладывается на время до первого токена и время генерации:

$$\ell_j \approx \alpha_s + \beta_s t_j^{\text{out}}, \quad \alpha_s \geq 0, \quad \beta_s \geq 0. \quad (9)$$

Неотрицательные ограничения обеспечивают физический смысл параметров. При прогнозе выходных токенов $\hat{o}_s(x)$ оставшаяся сервисная нагрузка равна

$$\hat{S}_{\text{rem}} = \sum_s \sum_{x=m_s+1}^{\lfloor \hat{M}_s \rfloor} \left(\hat{\alpha}_s + \hat{\beta}_s \hat{o}_s(x) \right) + \hat{S}_{\text{nonLLM}}. \quad (10)$$

Сервисное время нельзя напрямую считать календарным, поскольку вызовы перекрываются. Для свежего окна W фактический параллелизм оценивается как

$$C_{\text{raw}} = \frac{\sum_{j \in W} \ell_j}{t_W^{\text{max}} - t_W^{\text{min}}}, \quad \hat{C}_{\text{ach}} = w_W C_{\text{raw}} + (1 - w_W) C_{\text{max}}, \quad w_W = \frac{|W|}{|W| + 80}. \quad (11)$$

Итоговый прогноз длительности имеет физически интерпретируемый вид

$$\hat{D}_N = D_{\text{elapsed}} + \frac{\hat{S}_{\text{rem}}}{\hat{C}_{\text{ach}}}. \quad (12)$$

Наблюдаемое время D_{elapsed} остаётся неизменным; корректируется только неизвестный остаток.

3.3 Калибровка и неопределённость

Для завершённого запуска r множитель точного остаточного прогноза на доле q можно восстановить как

$$m_r^*(q) = \frac{D_r - D_{r,\text{elapsed}}(q)}{\hat{D}_r(q) - D_{r,\text{elapsed}}(q)}. \quad (13)$$

Калибровочная кривая $m(q) = \text{median}_r m_r^*(q)$ применяется только к хвосту:

$$\hat{D}^{\text{cal}}(q) = D_{\text{elapsed}}(q) + m(q)[\hat{D}(q) - D_{\text{elapsed}}(q)]. \quad (14)$$

Опорные множители равны 1,90 на 10%, 1,22 на 25%, 1,03 на 50% и 1,00 при завершении. Интервалы неопределённости строятся бутстрэпом логарифмических остатков e_i : для каждой повторной выборки пересчитываются хвост, сервисное время и длительность, после чего используются эмпирические квантили.

4 Реализованная система

Телеметрия агрегирует вызовы БЯМ по попыткам и стадиям и учитывает полный логический вызов, включая повторы структурированного вывода и переходы между моделями. Это обеспечивает совпадение итоговой токеной суммы с биллингом. Оценка параллелизма строится по временным меткам фактических модельных вызовов, а не по числу одновременно диспетчеризуемых мутантов.

Диагностический агент классифицирует изменение режима как **servers**, **programs**, **task** или **noise**. Агент пробуждается при выходе наблюдений за интервал либо при расхождении независимо построенных токенового и временного прогнозов. Его действие применяется только к будущему хвосту.

Механизм replay повторно выполняет оценщик и агент на фиксированном потоке событий. Типичное агентное воспроизведение занимает около 16 секунд и требует пяти модельных вызовов. Сравнимые политики получают одинаковые события, что позволяет отделить эффект политики от случайности эволюционного запуска.

5 Экспериментальная методика

Детерминированный оценщик проверялся на восьми завершённых исторических логах семейств Adversarial Code, AlgoTune и Kadane. Состояние восстанавливалось на фиксированной доле прогресса с использованием только уже доступных событий. Основная метрика — медианная абсолютная процентная ошибка:

$$\text{APE}_r(q) = 100 \frac{|\hat{D}_r(q) - D_r|}{D_r}, \quad \text{MdAPE}(q) = \text{median}_r \text{APE}_r(q). \quad (15)$$

Парная абляция агента включала 16 запусков на восьми задачах AlphaEvolve. Одна пара не дала пригодных точек, поэтому анализ выполнен на семи завершённых парах. Для задачи r сравнивалась разность

$$\delta_r(q) = \text{APE}_r^{\text{agent}}(q) - \text{APE}_r^{\text{auto}}(q). \quad (16)$$

Использовался двусторонний точный критерий Уилкоксона. Для разведочного анализа четырёх окон пробуждений применена поправка Бонферрони:

$$p_{\text{adj}} = \min(4p_{\text{raw}}, 1) = \min(4 \cdot 0,015, 1) \approx 0,060. \quad (17)$$

Порог значимости зафиксирован как $\alpha = 0,05$.

6 Результаты

6.1 Точность оценщика

Стадийная экстраполяция, поправка Дюана, полный учёт модельных вызовов, физические ограничения задержки и измеряемый параллелизм существенно улучшили прогноз на основных рабочих точках.

Таблица 1: Медианная абсолютная процентная ошибка до и после итоговой реализации.

Контрольная точка	Токены: до → после	Длительность: до → после
25% прогресса	25% → 18%	21% → 13%
50% прогресса	20% → 15%	37% → 11%
Завершение	22% → 7%	23% → 0,3%

Измеренный параллелизм менялся от 4,2 до 14,5 одновременных вызовов БЯМ. Связь между ним и временной ошибкой непосредственно подтверждает необходимость формулы (11).

Остаточная терминальная токеническая ошибка совпадает с долей расхода, отсутствующего в старых событиях LLM_CALL. Запуски с полным покрытием имеют близкую к нулю ошибку, а при пропуске 13–17% расхода сохраняется расхождение того же порядка.

6.2 Агент на задачах AlphaEvolve

Таблица 2: Парная абляция на семи завершённых задачах AlphaEvolve.

Прогресс	Автоматически	С агентом	Агент лучше	Парное p
10%	42,5%	41,8%	6/7	1,000
25%	30,1%	27,1%	5/7	0,453
50%	11,9%	12,3%	4/7	1,000
100%	0,7%	0,7%	5/7	0,453



Рис. 1: Фактический параллелизм по логам и ошибка длительности при использовании постоянного делителя.

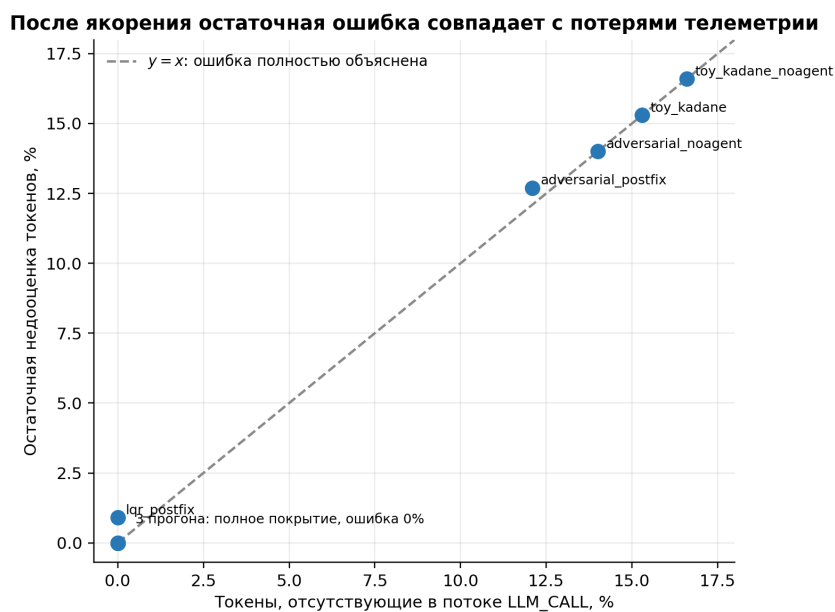


Рис. 2: Остаточное занижение токенов в зависимости от полноты событий.

На уровне задач подтверждающего преимущества агента не обнаружено. На 25% прогресса медианная ошибка ниже на 3,0 процентного пункта, однако парное $p = 0,453$. Для окна в 10 сбросов разведочный анализ пробуждений дал исходное $p = 0,015$ и $p \approx 0,060$ после поправки за четыре окна. Результат рассматривается как гипотеза для независимого исследования.

На уровне задач преимущество агента в чистой абляции AlphaEvolve не подтверждено

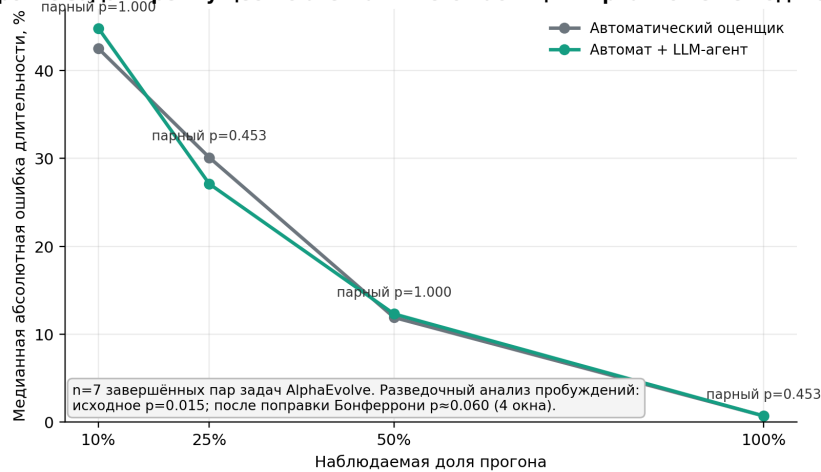


Рис. 3: Парная абляция на AlphaEvolve: подтверждённого эффекта на уровне задач нет; скорректированный разведочный результат равен $p \approx 0,060$.

7 Заключение

Проект завершён созданием сквозной системы онлайн-оценки стоимости эволюционного программного поиска с БЯМ. Детерминированная модель объединяет робастный прогноз стадийных токенов, калиброванное сервисное время и фактический параллелизм. На историческом корпусе она заметно улучшает среднесрочные и итоговые оценки. Replay-инфраструктура позволяет быстро и воспроизводимо сравнивать агентные политики.

Парная абляция AlphaEvolve пока не подтверждает преимущество диагностического агента на уровне задач. Значение $p \approx 0,060$ после поправки используется только как основание для следующего заранее спланированного эксперимента. Таким образом, подтверждённым практическим результатом является детерминированный оценщик, а агент остаётся исследуемым расширением.

Список литературы

- [1] B. Romera-Paredes, M. Barekatin, A. Novikov, et al., “Mathematical discoveries from program search with large language models,” *Nature*, vol. 625, pp. 468–475, 2024. doi:10.1038/s41586-023-06924-6.
- [2] A. Novikov et al., “AlphaEvolve: A coding agent for scientific and algorithmic discovery,” arXiv:2506.13131, 2025. <https://arxiv.org/abs/2506.13131>.
- [3] V. Khurlov, A. Galichin, D. Bashkurov, et al., “GigaEvo: An open source optimization framework powered by LLMs and evolution algorithms,” arXiv:2511.17592v1, 2025. <https://arxiv.org/abs/2511.17592v1>.
- [4] T. Jansen and C. Zarges, “Fixed-budget analysis in evolutionary computation,” in *Proceedings of GECCO*, 2012. doi:10.1145/2330163.2330347.
- [5] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, “Hyperband: A novel bandit-based approach to hyperparameter optimization,” *Journal of Machine Learning Research*, vol. 18, no. 185, pp. 1–52, 2018. <https://www.jmlr.org/papers/v18/16-558.html>.
- [6] P. K. Sen, “Estimates of the regression coefficient based on Kendall’s tau,” *Journal of the American Statistical Association*, vol. 63, no. 324, pp. 1379–1389, 1968. doi:10.1080/01621459.1968.10480934.
- [7] N. Duan, “Smearing estimate: A nonparametric retransformation method,” *Journal of the American Statistical Association*, vol. 78, no. 383, pp. 605–610, 1983. doi:10.1080/01621459.1983.10478017.

- [8] J. D. C. Little, “A proof for the queuing formula: $L = \lambda W$,” *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.
[doi:10.1287/opre.9.3.383](https://doi.org/10.1287/opre.9.3.383).